

資工系大三「軟體品質保證與測試 (SQA)」大破大立改革方案

—— 從品質模型到 AI 時代的軟體可靠性工程 (Software Quality Models & Reliability Engineering)

核心願景：在 2026 年，「寫程式碼 (Coding)」的成本已大幅降低，而「定義好軟體的標準 (Quality Models)」與「驗證程式碼 (Verification & Reliability)」成為人類工程師最無可取代的核心能力。本方案跳脫 1990~2010 年代的「老派品管流程與手動測試」框架，以現代軟體品質模型 (ISO 25010 / Garvin) 為指北針，全面轉型為：《AI 時代的軟體品質模型、可靠性與 AI 驅動測試工程 (Software Quality Models & AI-Driven Quality Engineering)》。

一、為什麼需要「大破大立」？（典範轉移）

1.1 傳統 SQA 教學的困境

傳統 SQA 教學模式 (舊思維)	學生實際感受 / 痛點
從教條式瀑布流程、CMMI 繁瑣等級開局	枯燥乏味，覺得離現代敏捷與 AI 開發極度遙遠
品質模型只當成名詞背誦，不知道與寫 Code 有何關係	考完就忘，寫程式時依然只管「能跑就好」
花數週講手動 Fagan 檢視會議、寫紙本 IEEE 829 表格	繁文縟節，學生覺得在做行政文書而非工程實踐
手工寫重複的 <code>assertEquals</code> 單元測試	學生直接讓 Copilot 秒生成，失去自己思考測資的動力
期末專題只是寫個 CRUD 系統附帶幾十個隨便寫的測試	缺乏鑑別度，無法驗證測試套件的真實防禦能力

1.2 2026 AI 時代的新定位：人類是「品質架構師與裁判」

- 寫 Code 很便宜，確保 Code 具備高品質極其昂貴：AI 雖然能秒寫程式碼，但它缺乏對整體品質模型（可靠性、可維護性、安全性、相容性、效能效率）的架構認知，產出的程式碼常充斥著隱蔽的邊界漏洞、競爭條件與幻覺。
- 品質模型是「指北針」，測試工程是「指揮 AI 的唯一韁繩」：
$$\text{品質模型 (ISO 25010)} \times \text{不變量測試 (Invariants)} \times \text{變異驗證 (Mutation Truth)} = \text{可信賴的高品質軟體}$$

二、五大核心改革支柱 (Five Radical Pillars)

- 五大改革支柱 (Pillars)
- ① 品質指北針：以 Garvin 五大觀點與 ISO 25010 模型定義「何謂好軟體」
 - ② 觀念重構：從「事後抓蟲」躍升至「不變量 (Invariants) 與屬性測試」
 - ③ 技術現代化：引入 Testcontainers、Pact 契約測試、Fuzzing
 - ④ AI 角色升級：培養「AI 測試架構師 (Agentic QA & Prompt for Spec)」
 - ⑤ 評量與專題顛覆：導入「紅藍軍品質攻防 (Red vs Blue) 與混沌工程」

支柱 ①：品質指北針 —— 建立「好軟體」的多維度認知 (Software Quality Models)

- 不再死背條文，而是建立工程視角：
 - David Garvin 的五大品質觀點：超自然（直覺美感）、使用者（需求滿足）、製造（標準合規）、產品（內在架構材質）、價值觀點（商業價值與成本）。
 - 現代品質模型標準 (從 ISO 9126 到 ISO 25010 SQuaRE)：
 - 8 大產品品質特性：功能適合性 (Functional Suitability)、效能效率 (Performance Efficiency)、相容性 (Compatibility)、易用性 (Usability)、可靠性 (Reliability)、安全性 (Security)、可維護性 (Maintainability)、

可移植性 (Portability)。

3. **品質特性與測試技術的完整映射**：讓學生清楚明白整學期學的每一項技術（JUnit, Mockito, PIT, JMeter, Fuzzing, Testcontainers, Playwright），究竟是在度量並守護品質模型的哪一個維度！

 品質模型 (ISO 25010) 與 實戰測試技術映射地圖

ISO 25010 品質特性	核心子特性 (Sub-characteristics)	本課程對應之測試與工程技術
功能適合性 (Functional Suitability)	功能完備性、功能正確性、功能適切性	等價類分割 (EP)、邊界值分析 (BVA)、JUnit 5、BDD (Cucumber)
可靠性 (Reliability)	成熟度、容錯性 (Fault Tolerance)、可回復性 (Recoverability)	斷言 (Assertions)、 屬性測試 (Property-Based Testing) 、混沌工程 (Chaos)
可維護性 (Maintainability)	模組化、可分析性、可修改性、可測試性	靜態程式碼分析 (SonarQube/SpotBugs)、 變異測試 (PITest) 、依賴解耦
安全性 (Security)	機密性、完整性、抗抵賴性、真實性	靜態安全掃描 (AST/SAST)、 模糊測試 (Fuzzing with Jazzer)
效能效率 (Performance Efficiency)	時間行為 (延遲/回應時間)、資源利用率、容量	k6 / Apache JMeter 高併發壓測、GC 監控與記憶體洩漏分析
相容性 (Compatibility)	共存性、互通性 (Interoperability)	微服務契約測試 (Pact) 、跨版本相容性測試
可移植性 (Portability)	適應性、易安裝性、易置換性	Testcontainers 容器化測試、雲原生多環境測試
易用性 (Usability)	易識別性、易學習性、易操作性、錯誤保護	Playwright E2E 驗收測試、使用者流程自動化驗證

支柱 ②：觀念重構 —— 從「單點測資」升級為「屬性與不變量 (Property-Based Testing)」

- **拋棄**：只教學生寫 `assert add(1, 2) == 3` 這種無聊的單點測資。
- **導入**：屬性測試 (**Property-Based Testing**，如 `jqwik` / `Hypothesis`)。
 - 教學生定義**系統不變量 (Invariants)** (例如：「任何陣列排序後長度與元素計數必不變」、「加密後再解密必等於原文」、「轉帳後雙方帳戶總額必守恒」)。
 - 讓測試框架自動亂數生成 **10,000 組極端邊界測資** 進行暴力破壞，並自動進行 **縮小化 (Shrinking)** 找出最小出錯案例！

支柱 ③：技術現代化 —— 捨棄老舊工具，導入雲原生與微服務測試

- **拋棄**：脆弱且速度慢的手動 Selenium 網頁測試、過時的純 Mock 架構。
- **導入**：
 - **Testcontainers**：在 Docker 容器中直接拉起真實的 PostgreSQL、Redis、Kafka 進行毫秒級整合測試。
 - **契約測試 (Contract Testing with Pact)**：現代微服務架構下，確保前後端或服務間 API 合約不被意外破壞的標準解法。
 - **模糊測試 (Fuzzing with Jazzer/AFL)**：自動生成非結構化、惡意、畸形輸入，主動挖掘記憶體溢位與崩潰漏洞。

支柱 ④：AI 角色升級 —— 學生扮演「Agentic QA 架構師」

- 不准學生當「只會按 Tab 的碼農」，而是教學生：

1. **規格先行 (Spec-Driven AI Development)**：用形式化規格引導 AI 生成極端邊界測試。
2. **AI 生成測試的毒性檢驗**：使用 **PIT 變異測試** 作為「照妖鏡」，客觀測量 AI 寫的測試是真有效還是假裝綠燈。
3. **自律型 QA Agent (Autonomous Testing Agents)**：學習構建能自動爬梳網站、自動探索未知 UI 路徑、自動回報重現步驟的 AI 測試機器人。

支柱 ⑤：專題與評量大破大立 —— 「紅藍軍攻防挑戰賽 (Chaos Arena)」

- **期末專題不再各自交報告，而是進行【紅白攻防對抗】：**
 - 藍軍（防守）：打造高可靠度系統，建立完整測試金字塔 + 屬性測試 + 變異防護 + 品質模型指標報告。
 - 紅軍（進攻）：使用 AI 模糊測試、混沌故障注入 (Chaos Monkey)、極端邊界攻擊對手的系統，看誰能攻破對手的防線！
-

三、大破大立版：16 週全新課程大綱 (The Modern Syllabus)

週次	核心模組	1 小時講授 (Lecture - 理論/前沿思維)	2 小時實習 (Lab - 現代工具實戰)	品質模型對應維度 / 產出
01	第一階段：品質模型與 AI 信任危機	【開局震撼】：為什麼 AI 生成的完美代碼會在第 101 小時崩潰？軟體危機、Garvin 五大品質觀點與軟體要素	AI 代碼破壞實驗：給定一個 AI 生成的交易系統，找出隱藏的並發、精度與崩潰漏洞	認識品質的代價與重要性
02	第一階段	【核心理論】現代軟體品質模型 (ISO 9126 $\rightarrow$ ISO 25010)：8 大產品品質特性、使用品質與品質成本 (CoQ 1:10:100 定律)	系統品質度量評估 (Quality Metrics)：使用工具量化分析開源專案之可靠性、可維護性與缺陷密度	建立「好軟體」的評鑑指北針
03	第一階段	防禦性架構與合約設計 (Design by Contract)：前置/後置條件、狀態不變量 (Class Invariants) 與自我診斷防線	使用 Java Assertions、Google Guava Preconditions 與結構化日誌 (SLF4J/MDC) 建立自我防護	守護：可靠性 (容錯度) Lab 01：防禦防線建置
04	第一階段	現代靜態分析與架構異味檢測：從 AST 語法樹看程式碼異味 (Code Smells)、安全漏洞 (OWASP Top 10) 與架構腐化	打造 CI 靜態品質門檻：SonarQube / SpotBugs / PMD 規則設定與自訂檢測規則	守護：可維護性、安全性 Lab 02：Code Smell 攔截
05	第二階段：測試理論的極致與現代化	黑箱設計之魂：等價分割、邊界分析、全成對測試 (Pairwise) 與正交表數學原理	JUnit 5 現代架構：動態測試 (<code>@TestFactory</code>)、參數化測試 (<code>@ParameterizedTest</code>) 與自訂 DisplayName	守護：功能適合性 專題分組與賽制發布
06	第二階段	【典範轉移】屬性基礎測試 (Property-Based Testing)：告別手寫測資，用數學屬性 (Invariants) 讓電腦自動生成萬組測資	<code>jqwik</code> / <code>Hypothesis</code> 實戰：定義演算法不變量，體驗框架自動產生極端測資與縮小化 (Shrinking)	守護：可靠性 (極限邊界) 顛覆傳統測試思維
07	第二階段	白箱測試與 MC/DC (Modified Condition/Decision Coverage)：航空級高可靠度軟體的覆蓋率標準與圈複雜度推導	JaCoCo 高級分析：分支與指令覆蓋率解讀、為未覆蓋路徑精準補彈	守護：功能適合性、結構完整 Lab 03：高強度白箱實戰
08	期中檢驗	【期中能力鑑定 (Midterm Exam)】(涵蓋 ISO 25010 品質模型、規格推導、MC/DC 分析、屬性不變量與測試設計)	【專題提案與藍軍品質架構審查 (Architecture & Spec Review)】：各組發表系統規格、防禦藍圖與品質指標目標	期中考 & 藍軍防禦架構確認
09	第三階段：微服務、真實	變異測試 (Mutation Testing) —— 測試品質的唯一真理：誰來監督監督者？變異算子、殺死率與等價變異體難題	PIT (Pitest) 實戰：注入故障變異體，計算 Mutation Score，揪出「高覆蓋率卻測不出 Bug」的假測試	守護：測試套件可維護性與有效性

週次	核心模組	1 小時講授 (Lecture - 理論/前沿思維)	2 小時實習 (Lab - 現代工具實戰)	品質模型對應維度 / 產出
	環境與非功能測試			Lab 04：變異殺死挑戰
10	第三階段	隔離架構與 Test Double 原則：何時用 Mock？何時不用 Mock？過度 Mock 的反模式與脆化測試 (Brittle Tests)	Mockito 進階 ：深入 <code>ArgumentCaptor</code> 、Spy、嚴格 Stubbing (<code>Strictness.STRICT_STUBS</code>) 與架構解耦	守護：可測試性、模組化 Lab 05：重構過度 Mock 代碼
11	第三階段	真實環境整合測試 (Testcontainers) 與 API 測試 ：拒絕 H2 內存庫幻覺，使用真實 Docker 容器測試資料庫與中介軟體	Testcontainers + Spring Boot Test ：一鍵拉起真實 PostgreSQL / Redis 容器進行毫秒級資料庫整合測試	守護：可移植性、環境一致性 Lab 06：容器化整合實戰
12	第三階段	微服務契約測試 (Contract Testing with Pact) & 現代 E2E ：分散式系統中 API 契約防護；Playwright 現代化 Web 測試	Pact 實戰 + Playwright 自動化 ：定義 Consumer-Driven Contracts；撰寫抗網路波動、具錄影回放的 Web 驗收測試	守護：相容性、易用性 Lab 07：契約與 E2E 防衛
13	第四階段：前沿品質工程、AI 與混沌實戰	高併發、效能與負載工程：TPS、P99 延遲、資源洩漏、排隊理論與壓測模型設計	k6 / JMeter 現代壓測 ：用程式碼定義負載情境 (Load as Code)，進行突波測試 (Spike) 與耐力測試 (Soak)	守護：效能效率 Lab 08：效能瓶頸診斷
14	第四階段	模糊測試 (Fuzzing) 與混沌工程 (Chaos Engineering)：注入混亂，在生產環境崩潰前主動搞壞系統	Chaos-Mesh / Fault Injection 實作 ：隨機注入網路延遲、殺死 Pod、模擬磁碟滿載，驗證系統容錯自癒力	守護：可靠性、安全性 Lab 09：混沌破壞實驗
15	第四階段	【AI in SQA 前沿】 Agentic Testing 與自律測試機器人：Prompt Engineering for QA、AI 輔助探索性測試、自我修復測試套件	打造 GitHub Actions 自動化 CI/CD 全防線 (包含 SonarQube + JaCoCo + PIT + AI 程式碼審查 Bot)	守護： DevOps 流程品質 藍軍系統封裝，紅軍備戰
16	決戰與收官	【期末能力筆試 (Final Exam)】(30% 觀念：ISO 25010 品質架構、微服務測試、混沌工程、AI 測試驗證準則)	🔥 【紅藍軍品質攻防大擂台 (Red vs Blue Chaos Arena)】 ：現場 Live 攻防滲透、展示自癒防線與頒獎	專題公開發表與競賽

✂ 四、顛覆性期末專題：【紅藍軍品質攻防擂台 (Red vs. Blue Arena)】

4.1 專案運作機制（賽事化）

專題不再是「各組做一個普通系統交報告」，而是將全班分為多個 **戰鬥小隊（每隊 3~4 人）**，進行兩階段的工程對抗：

【階段一：藍軍築防（第 08~14 週）】 打造符合 ISO 25010 高可靠微服務系統 • 完整的規格定義（OpenAPI / Invariants） • Property-Based Testing + PIT > 75% • Testcontainers + Playwright E2E • 完善的 CI/CD Quality Gate 自動防線 • 產出《ISO 25010 品質模型達成度度量報告》	【階段二：紅藍交鋒（第 15~16 週）】 各隊互換權限，扮演「紅軍攻擊者」 • 使用 AI 模糊測試（Fuzzing）注入極端測試 • 注入混沌故障（Chaos/Concurrency/Memory） • 尋找防守方未考慮的邊界漏洞與當機條件 • 撰寫「漏洞滲透報告」並提交 Issue
---	---

4.2 專題評分規準 (Rubrics, 滿分 100%)

評分維度	權重	具體評審重點與標準	對應品質模型特性
1. 藍軍系統品質模型達成度	25%	是否提出具體的《ISO 25010 品質度量矩陣》？架構是否具備高內聚低耦合、異常保護與結構化日誌？	可維護性、可分析性
2. 測試深度與變異殺死強度	25%	屬性測試 (Property-Based) 涵蓋度；JaCoCo 覆蓋率（行 $\geq 80\%$ 、分支 $\geq 75\%$ ）；PIT 變異殺死率（ $\geq 70\%$ ）。	可靠性 (容錯度)、功能適合性
3. 真實容器化與 CI/CD 防線	20%	Testcontainers 真實資料庫整合；Playwright E2E 測試；GitHub Actions 自動阻擋不合格 PR。	可移植性、相容性
4. 紅軍攻擊深度與滲透報告	20%	攻擊腳本精妙度（是否運用模糊測試、極端併發競爭條件、邊界值）；提交給對手的 Bug Report 是否專業且可重現。	安全性、健壯性檢驗
5. AI 賦能與現場 Demo 答辯	10%	攻防 Live Demo 流暢度、如何批判性使用 AI 工具進行測試生成與變異驗證。	綜合工程素養

🎯 五、結論：這門課將為資工系大三學生帶來的終生競爭力

這套大破大立的課程設計，以**軟體品質模型**為靈魂、以**現代可靠性工程**為骨架、以**AI 賦能**為翅膀，讓資工系大三學生掌握頂級實戰力：

1. **格局與視野**：不再只是「碼農」，而是懂得從 **Garvin 五大觀點與 ISO 25010 八大維度** 評鑑與架構系統的「品質工程師」。
2. **頂尖技術鏈**：精通 Testcontainers、jqwik、PIT、Playwright、Pact、k6、GitHub Actions 等一線現代工具。
3. **AI 時代不可替代性**：成為能夠駕馭 AI、定義嚴密規格、並以數學屬性與變異防線驗證 AI 產出的「現代軟體可靠性架構師」。